

Lekcja 13 – [ZASTĘPSTWO] TERMIN ODDANIA PROJEKTÓW

Commits on Feb 23, 2026

Temat: Brak modyfikatorów dostępu

W Pythonie nie ma prawdziwych modyfikatorów dostępu (jak *public*, *private*, *protected* w *Java*/*C++*/*C#*). Nie istnieją żadne słowa kluczowe do tego celu.

Python używa tylko **konwencji nazewnictwa** (i jednego mechanizmu *name mangling*). Są dokładnie trzy poziomy:

Poziom	Zapis	Co oznacza	Czy naprawdę chroniony?
Public	<code>self.nazwa</code>	Domyślny – każdy może czytać i modyfikować	Nie – całkowicie otwarty
Protected	<code>self._nazwa</code>	Tylko konwencja („nie ruszaj z zewnątrz klasy”)	Nie – dostępny zawsze
Private	<code>self.__nazwa</code>	Name mangling + konwencja	Prawie tak (ale da się obejść)

- **Public** – bez podkreślnika
Wszystko, co nie ma `_` na początku, jest publiczne.
- **Protected** – pojedynczy `_`
Tylko sygnalizuje innym programistom: „to jest wewnętrzne, lepiej nie używaj bezpośrednio”. Python **nie blokuje** dostępu. W Pythonie **protected** to **tylko konwencja**, nie prawdziwa ochrona.
- **Private** – dwa `_`
Python automatycznie zmienia nazwę na `_NazwaKlasy__nazwa` (*name mangling*). Dzięki temu nie da się przypadkowo odwołać z zewnątrz, ale **nadal można** dostać się przez `_Klasa__nazwa`. To jest **najmocniejsza forma "prywatności"** jaką Python oferuje. Python robi coś specjalnego nazywanego **name mangling** („przekształcanie nazwy”).

Przykład:

```
class Osoba:
    def __init__(self, imie, wiek):
        self.imie = imie          # ← PUBLICZNA (self.nazwa)
        self._wiek = wiek        # protected (tylko dla porównania)
        self.__haslo = "sekret"  # private (tylko dla porównania)

    def przedstaw_sie(self):
        print(f"Cześć, jestem {self.imie}")
```

```
class Raport:

    def generuj_raport(self, osoba: Osoba):

        print(f"RAPORT:")
        print(f"    Imię: {osoba.imie}")          # ← działa zawsze
        # print(osoba._wiek)          # działa, ale nie powinieneś
        # print(osoba.__haslo)        # AttributeError
```

```
class Pracownik(Osoba):
    def __init__(self, imie, wiek, stanowisko):
        super().__init__(imie, wiek)
        self.stanowisko = stanowisko

    def pokaz_dane(self):

        print(f"Pracownik: {self.imie}")
        print(f"Stanowisko: {self.stanowisko}")
```

```
jan = Osoba("Jan Kowalski", 35)
anna = Pracownik("Anna Nowak", 28, "programista")
```

```
raport = Raport()
raport.generuj_raport(jan)
raport.generuj_raport(anna)
```

```
anna.pokaz_dane()
```

```
print(f"Bezpośrednio: {jan.imie}")
```

Dlaczego nie stosować **Protected** – pojedynczy _

1. Wyobraź sobie, że klasa Osoba jest napisana przez kogoś innego (np. bibliotekę, kolegę z zespołu, albo Ciebie sprzed 6 miesięcy).

```
class Osoba:
    def __init__(self, imie, wiek):
        self.imie = imie
        self._wiek = wiek          # ← PROTECTED (tylko konwencja!)

    def urodziny(self):
        self._wiek += 1

    def pokaz_wiek(self):
        print(f"{self.imie} ma {self._wiek} lat")
```

2. Co się stanie, jeśli Ty użyjesz osoba._wiek z zewnątrz?

```

jan = Osoba("Jan", 30)
print(jan._wiek)          # 30 ← działa
jan._wiek = 100           # zmieniamy bezpośrednio ← ZŁE!

```

To działa... ale jest **niebezpieczne**.

3. Za miesiąc autor klasy Osoba robi update:

```

class Osoba:                                # nowa wersja
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # ← nadal protected
        # self._wiek już NIE ISTNIEJE!

    def urodziny(self):
        print(f"🎉 Wszystkiego najlepszego {self.imie}!")

    @property
    def wiek(self):
        today = date.today()
        wiek = today.year - self._data_urodzenia.year
        if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):
            wiek -= 1
        return wiek

```

Co się stało z kodem, który używał jan._wiek?

```

jan = Osoba("Jan", date(1995, 3, 15))
print(jan._wiek)          # ← AttributeError! Nie ma już takiej zmiennej!
jan._wiek = 50            # ← też błąd!

```

4. TAK POWINNO SIĘ ROBIĆ (najlepsza praktyka 2025)

```

from datetime import date

class Osoba:
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # protected – tylko wewnątrz

    def urodziny(self):
        print(f"🎉 Wszystkiego najlepszego {self.imie}!")

    # ===== PUBLICZNY INTERFEJS =====
    @property
    def wiek(self):                                # getter – tylko odczyt
        """Zwraca aktualny wiek"""
        today = date.today()
        wiek = today.year - self._data_urodzenia.year
        if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):

```

```

        wiek -= 1
    return wiek

@wiek.setter
def wiek(self, nowy_wiek):          # setter – bezpieczna zmiana
    if nowy_wiek < 0:
        raise ValueError("Wiek nie może być ujemny!")
    # przeliczamy datę urodzenia na podstawie nowego wieku
    today = date.today()
    self._data_urodzenia = date(today.year - nowy_wiek, today.month, today.day)

jan = Osoba("Jan Kowalski", date(1995, 3, 15))

print(jan.wiek)                    # 30 (lub aktualny wiek) ← OK
jan.wiek = 31                      # zmienia wiek bezpiecznie ← OK
jan.urodziny()

# jan._data_urodzenia              # ← działa technicznie, ale NIE RÓB TEGO!

```

Przykład Private – dwa podkreślniki __

```

class KontoBankowe:
    def __init__(self, wlasciciel):
        self.wlasciciel = wlasciciel
        self.__saldo = 5000          # ← PRIVATE (dwa __)

    def wplac(self, kwota):
        self.__saldo += kwota
        print(f"Wpłacono {kwota} zł")

    def wypłac(self, kwota):
        if kwota <= self.__saldo:    # używamy wewnątrz klasy – OK
            self.__saldo -= kwota
            print(f"Wypłacono {kwota} zł")
        else:
            print("Brak środków!")

    def pokaz_saldo(self):
        print(f"Saldo: {self.__saldo} zł")    # też wewnątrz klasy

konto = KontoBankowe("Jan Kowalski")

konto.wplac(1000)
konto.pokaz_saldo()                 # działa → Saldo: 6000 zł

# Próba bezpośredniego dostępu (to, co większość ludzi próbuje):
print(konto.__saldo)                # ← AttributeError: 'KontoBankowe' object has no attribute
'__saldo'

```

```
# Próba zmiany:  
# konto.__saldo = 999999      # też nie działa tak, jak myślisz
```

Jak obejść name mangling?

```
# Prawdziwa nazwa po manglingu:  
print(konto._KontoBankowe__saldo)    # ← działa! → 6000  
  
# Możemy nawet zmienić:  
konto._KontoBankowe__saldo = 100     # teraz saldo = 100 zł  
konto.pokaz_saldo()                   # → Saldo: 100 zł
```

Lekcja 14 **TERMIN ODDANIA PROJEKTÓW** Commits on Feb 23, 2026

Temat: Obsługa plików: Otwieranie (with open), czytanie/zapis (read, write), tryby (r, w, a).

Obsługa plików

Python ma bardzo prosty i **bezpieczny sposób pracy z plikami** dzięki wbudowanej funkcji `open()` oraz **menedżerowi kontekstu `with`**. Dzięki temu **nie musisz pamiętać o ręcznym zamykaniu pliku (`close()`)**, a kod jest czytelniejszy i mniej podatny na błędy.

1. Otwieranie pliku – with open(...)

Zalecany sposób:

```
with open('nazwa_pliku.txt', 'tryb') as plik:  
    # tutaj pracujemy z plikiem  
    # po wyjściu z bloku with plik jest automatycznie zamykany
```

with

- Automatycznie zamyka plik nawet jeśli wystąpi błąd (wyjątek).
- Kod jest krótszy i czytelniejszy.
- Nie musisz pamiętać o `plik.close()`.

2. Tryby otwierania plików (mode)

'r' – read (domyślny)

Otwiera plik tylko do odczytu. Plik **musi istnieć**, inaczej dostaniesz błąd.

'w' – write

Tworzy nowy plik lub **całkowicie nadpisuje** istniejący.

'a' – append

Dopisuje dane na końcu pliku. Jeśli plik nie istnieje – automatycznie go tworzy.

Dodatkowo często spotykane:

- 'r+' – odczyt + zapis
- 'w+' – odczyt + zapis (nadpisuje)
- 'a+' – odczyt + dopisywanie

Przykład otwierania w różnych trybach:

```
with open('dane.txt', 'r') as plik:    # tylko odczyt
    ...

with open('wyniki.txt', 'w') as plik:  # nadpisanie
    ...

with open('log.txt', 'a') as plik:     # dopisywanie
    ...
```

3. Czytanie pliku – metody read(), readline(), readlines()

Przykład - czytanie całego pliku:

```
with open('dane.txt', 'r', encoding='utf-8') as plik:
    # 1. Odczyt całego pliku naraz (najprostsze)
    zawartosc = plik.read()
    print(zawartosc)

    plik.seek(0) # wróc na początek pliku
    # 2. Odczyt linia po linii (najlepsze dla dużych plików)
    for linia in plik:
        print(linia.strip())          # strip() usuwa \n na końcu

    plik.seek(0) # wróc na początek pliku
    # 3. Odczyt jako lista linii
    linie = plik.readlines()
    print(linie)
```

Praktyczne przykłady zastosowania:

```
'''
Zawartość pliku config.txt:
host = localhost
port = 8080
username = admin
password = sekret
debug = true
timeout = 30
'''

# Przykład 1: Wczytanie konfiguracji
with open('config.txt', 'r', encoding='utf-8') as plik:
    for linia in plik:
        if '=' in linia:
            klucz, wartosc = [x.strip() for x in linia.split('=', 1)]
            print(f"{klucz} = {wartosc}")
'''

Zawartość pliku zakupy.txt:
mleko
chleb
jajka
masło
ser
'''

# Przykład 2: Wczytanie listy zakupów do listy Pythona
with open('zakupy.txt', 'r', encoding='utf-8') as plik:
    lista_zakupow = [linia.strip() for linia in plik if linia.strip()]
    print(lista_zakupow)
```

4. Zapis pliku – metoda write() i writelines()

```
# Tryb 'w' → nadpisuje plik
with open('wyniki.txt', 'w', encoding='utf-8') as plik:
    plik.write("Wynik obliczeń: 42\n")
    plik.write("Druga linia tekstu\n")

# Można też zapisać wiele linii naraz
linie = ["Pierwsza\n", "Druga\n", "Trzecia\n"]
plik.writelines(linie)
```

Zapisać plik wyniki.txt z taką zawartością:

Wynik obliczeń: 42

Druga linia tekstu

Pierwsza

Druga

Trzecia

```
# Tryb 'a' → dopisuje na końcu (idealny do logów)
```

```
import datetime

with open('log.txt', 'a', encoding='utf-8') as plik:
    czas = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    plik.write(f"[{czas}] Użytkownik załogował się\n")
    plik.write(f"[{czas}] Błąd: coś poszło nie tak\n")
```

Praktyczne przykłady zastosowania:

```
import datetime
```

```
# Przykład 1: Zapis listy zakupów (nadpisanie)
zakupy = ["mleko", "chleb", "masło", "jajka"]

with open('lista_zakupow.txt', 'w', encoding='utf-8') as plik:
    for produkt in zakupy:
        plik.write(produkt + '\n')
```

```
# Przykład 2: Logowanie błędów (dopisywanie)
try:
    1 / 0
except Exception as e:
    with open('errors.log', 'a', encoding='utf-8') as plik:
        plik.write(f"[{datetime.datetime.now()}] BŁĄD: {e}\n")
```

Wynik:
Utworzy plik `lista_zakupow.txt` z zawartością:
mleko
chleb
masło
jajka

Lekcja 15

Temat: Zastosowania modułów i pakietów: Import, from-import, tworzenie własnych modułów, init.py

Moduł to po prostu plik z rozszerzeniem `.py`, który zawiera kod Pythona (funkcje, klasy, zmienne, instrukcje).

Dzięki modułom możesz podzielić swój kod na mniejsze, logiczne części i używać go wielokrotnie w innych plikach.

1. Importowanie modułu – import

```
import math
import random
import os

print(math.pi)
print(math.sqrt(16))

liczba = random.randint(1, 10)
print(liczba)

print(f"Aktualny katalog roboczy: {os.getcwd()}")
```

2. Importowanie konkretnych rzeczy – from ... import ...

```
from math import sqrt, pi
from random import randint, choice
```



```
from os import getcwd

print(sqrt(25))
print(pi)

losowa = randint(1, 100)
print(losowa)

print(f"Aktualny katalog roboczy: {getcwd()}")
```

Możesz też importować wszystko (niezalecane w dużych projektach):

```
from math import *
```

3. Tworzenie własnych modułów

Przykład struktury:

```
moj_projekt/
├── main.py
├── obliczenia.py
└── utils.py
```

Plik obliczenia.py (moduł):

```
# obliczenia.py

def dodaj(a, b):
    return a + b

def odejmij(a, b):
    return a - b

def pomnoz(a, b):
    return a * b

# zmienne na poziomie modułu
PI = 3.14159
TAX_RATE = 0.23

class Kalkulator:
    def __init__(self):
        self.wynik = 0

    def dodaj(self, x):
        self.wynik += x
        return self.wynik
```

Plik main.py (używamy modułu):

```
import obliczenia
from obliczenia import dodaj, Kalkulator

print(obliczenia.dodaj(5, 7))           # 12
print(dodaj(10, 20))                   # 30

kalk = Kalkulator()
print(kalk.dodaj(100))                  # 100
print(kalk.dodaj(50))                   # 150
```

4. Pakiety (packages) – struktura z wieloma modułami

Pakiet to folder zawierający plik `__init__.py` (w Python 3.3+ nie jest już obowiązkowy, ale nadal bardzo zalecany).

Przykład struktury pakietu:

```
sklep/
├── __init__.py
├── produkty.py
├── zamowienia.py
├── platnosci/
│   ├── __init__.py
│   ├── karta.py
│   └── przelew.py
└── utils/
    ├── __init__.py
    └── helpers.py
```

Plik sklep/produkty.py:

```
# sklep/produkty.py

class Produkt:
    def __init__(self, nazwa, cena):
        self.nazwa = nazwa
        self.cena = cena

def stworz_produkt(nazwa, cena):
    return Produkt(nazwa, cena)
```

Plik sklep/zamowienia.py:

```
# sklep/zamowienia.py

from .produkty import Produkt  # import względny
```

```
class Zamowienie:
    def __init__(self):
        self.produkty = []

    def dodaj_produkt(self, produkt):
        self.produkty.append(produkt)
```

Plik sklep/__init__.py

```
# sklep/__init__.py

# Możemy tutaj wystawić wygodny interfejs publiczny pakietu

from .produkty import Produkt, stworz_produkt
from .zamowienia import Zamowienie

# Możemy też wykonać kod inicjalizujący pakiet
print("Pakiet 'sklep' został zaimportowany")

__all__ = ['Produkt', 'stworz_produkt', 'Zamowienie']
```

Użycie pakietu w main.py:

```
# main.py

import sklep
from sklep import Produkt, stworz_produkt, Zamowienie

p1 = stworz_produkt("Laptop", 4999)
p2 = Produkt("Myszka", 89)

zam = Zamowienie()
zam.dodaj_produkt(p1)
zam.dodaj_produkt(p2)

print(f"Złożono zamówienie na {len(zam.produkty)} produktów")
```

6. Rola pliku __init__.py

Plik __init__.py ma kilka ważnych funkcji:

1. **Zamienia folder w pakiet** (Python wie, że to nie zwykły folder)
2. **Kod inicjalizujący** – wykonuje się przy pierwszym imporcie pakietu
3. **Definiowanie __all__** – kontroluje co jest importowane przy from pakiet import *
4. **Wystawianie API** – wygodne importy z poziomu pakietu

Przykład zaawansowanego __init__.py:

```
# sklep/__init__.py
```

```
from .produkty import Produkt, stworz_produkt
from .zamowienia import Zamowienie
from .platnosci.karta import PlatnoscKarta

# Możesz stworzyć aliasy
from .utils.helpers import formatuj_cene as fc

__version__ = "1.2.3"
__all__ = ['Produkt', 'Zamowienie', 'stworz_produkt', 'PlatnoscKarta']
```

Dzięki temu użytkownik pakietu może pisać:

```
from sklep import Produkt, Zamowienie
# zamiast:
# from sklep.produkty import Produkt
# from sklep.zamowienia import Zamowienie
```

Lekcja 16 [ZASTĘPSTWO]

Temat: Polimorfizm i enkapsulacja

Python jest językiem obiektowym i w pełni wspiera polimorfizm oraz enkapsulację (hermetyzację).

Polimorfizm w Pythonie to zdolność obiektów różnych klas do reagowania na te same metody w sposób odpowiedni dla ich typu. Oznacza to, że ta sama operacja może zachowywać się inaczej w zależności od typu obiektu, na którym jest wykonywana.

1. Polimorfizm funkcji wbudowanych

```
# Funkcja len() działa z różnymi typami
print(len("Hello")) # 5 (string)
print(len([1, 2, 3, 4])) # 4 (lista)
print(len({'a': 1, 'b': 2})) # 2 (słownik)
```

2. Polimorfizm z klasami i metodami

Polimorfizm - ta sama metoda, różne zachowania

```
class Pies:
    def dzwiek(self):
        return "Hau hau!"

    def opis(self):
        return "Jestem psem"
```

```
class Kot:
    def dzwiek(self):
        return "Miau!"
    def opis(self):
        return "Jestem kotem"
```

```
class Krowa:
    def dzwiek(self):
        return "Muuu!"
    def opis(self):
        return "Jestem krową"
```

```
def wydaj_dzwiek(zwierze):
    print(zwierze.dzwiek())
```

```
pies = Pies()
kot = Kot()
krowa = Krowa()
```

```
wydaj_dzwiek(pies)
wydaj_dzwiek(kot)
wydaj_dzwiek(krowa)
```

3. Polimorfizm z dziedziczeniem

1. Przykład

```
class Figura:
    def pole(self):
        pass

    def obwod(self):
        pass

class Prostokat(Figura):
    def __init__(self, szerokosc, wysokosc):
        self.szerokosc = szerokosc
        self.wysokosc = wysokosc

    def pole(self):
        return self.szerokosc * self.wysokosc

    def obwod(self):
        return 2 * (self.szerokosc + self.wysokosc)

class Kolo(Figura):
    def __init__(self, promien):
        self.promien = promien

    def pole(self):
        return 3.14 * self.promien ** 2

    def obwod(self):
        return 2 * 3.14 * self.promien

class Trojkat(Figura):
    def __init__(self, a, b, c):
```

```

        self.a = a
        self.b = b
        self.c = c

    def pole(self):
        # Wzór Herona
        s = (self.a + self.b + self.c) / 2
        return (s * (s - self.a) * (s - self.b) * (s - self.c)) ** 0.5

    def obwod(self):
        return self.a + self.b + self.c

# Użycie polimorfizmu
figury = [ Prostokat(5, 10), Kolo(7), Trojkat(3, 4, 5) ]
for figura in figury:
    print(f"Pole: {figura.pole():.2f}")
    print(f"Obwód: {figura.obwod():.2f}")
    print("---")

```

Wyjście:

```

Pole: 50.00
Obwód: 30.00
---
Pole: 153.86
Obwód: 43.96
---
Pole: 6.00
Obwód: 12.00
---

```

2. Przykład

```

class Kształt:
    def __init__(self, nazwa= " Kształt "):
        self.nazwa = nazwa

    def pole(self):
        print(" Brak danych na temat ", self.nazwa)

class Trojkat(Kształt):
    def __init__(self, nazwa= " Trojkat ", a=2, h=2):
        super().__init__(nazwa)
        self.a = a
        self.h = h

    def pole(self):
        print(" Pole figury ", self.nazwa)
        print(self.a* self.h/2)

class Prostokat(Kształt):
    def __init__(self, nazwa= " Prostokat ", a=2, b=2):
        super().__init__(nazwa)
        self.a = a
        self.b = b

    def pole(self):
        print(" Pole figury ", self.nazwa)
        print(self.a* self.b)

class Kwadrat():
    def __init__(self, nazwa="Kwadrat", a=3):

```

```
self.nazwa = nazwa
self.a = a
```

```
def pole(self):
    print("Pole figury ", self.nazwa)
    print(self.a**2)
```

```
def wyswietl_pole(lista):
    for x in lista:
        x.pole()
```

```
ksztalt = Kszalt()
trojkat = Trojkat()
prostokat = Prostokat()
kwadrat = Kwadrat()
```

```
for x in (ksztalt, trojkat, prostokat, kwadrat):
    print(type(x))
    x.pole()
```

4. Polimorfizm z operatorami (przeciążanie operatorów)

Przeciążenie operatorów (Operator Overloading)

Przeciążenie operatorów to mechanizm, dzięki któremu możemy **zmienić zachowanie** standardowych operatorów (+, -, *, /, ==, >, [] itd.) dla naszych własnych klas.

Dzięki temu zamiast pisać:

```
v3 = v1.add(v2)
```

możemy pisać naturalnie:

```
v3 = v1 + v2
```

Python umożliwia to poprzez **specjalne metody** (tzw. **dunder methods** – double underscore), które zaczynają i kończą się na `__`.

Przykład:

```
class Wektor:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        # Przeciążenie operatora +
        return Wektor(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Wektor({self.x}, {self.y})"

    def __mul__(self, scalar):
        # Mnożenie przez skalar
        return Wektor(self.x * scalar, self.y * scalar)
```

```
v1 = Wektor(2, 3)
v2 = Wektor(5, 7)
```

`v3 = v1 + v2` # Używamy przeciążonego operatora +

```
print(v3) # Wektor(7, 10)
print(v1 * 3) # Wektor(6, 9)
```

Pod spodem

- `v1 + v2` → Python szuka metody `__add__` w klasie `Wektor`
- `print(v3)` → Python szuka metody `__str__`
- `v1 * 3` → Python szuka metody `__mul__`

Najważniejsze metody do przeciążania operatorów

Operator	Metoda specjalna	Przykład użycia
+	<code>__add__</code>	<code>v1 + v2</code>
-	<code>__sub__</code>	<code>v1 - v2</code>
*	<code>__mul__</code>	<code>v1 * 3</code>
/	<code>__truediv__</code>	<code>v1 / 2</code>
//	<code>__floordiv__</code>	<code>v1 // 2</code>
==	<code>__eq__</code>	<code>v1 == v2</code>
!=	<code>__ne__</code>	<code>v1 != v2</code>
>	<code>__gt__</code>	<code>v1 > v2</code>
<	<code>__lt__</code>	<code>v1 < v2</code>
>=	<code>__ge__</code>	<code>v1 >= v2</code>
<=	<code>__le__</code>	<code>v1 <= v2</code>
<code>len()</code>	<code>__len__</code>	<code>len(v1)</code>
<code>str()</code> / <code>print</code>	<code>__str__</code>	<code>print(v1)</code>
<code>repr()</code>	<code>__repr__</code>	<code>repr(v1)</code>
<code>[]</code>	<code>__getitem__</code>	<code>v1[0]</code>

5. Polimorfizm operatorów

Przykład

```
print(2 + 3) # 5
print("2" + "3") # "23"
```

Główne zastosowania polimorfizmu:

1. **Uproszczenie kodu** - jedna funkcja obsługuje wiele typów obiektów
2. **Rozszerzalność** - łatwo dodawać nowe typy bez zmiany istniejącego kodu
3. **Abstrakcja** - ukrywanie szczegółów implementacji
4. **Zgodność z zasadą Open/Closed** - otwarty na rozszerzenia, zamknięty na modyfikacje
5. **Czytelność** - kod jest bardziej intuicyjny i łatwiejszy do zrozumienia

Enkapsulacja w Pythonie to mechanizm ukrywania wewnętrznych szczegółów implementacji klasy i udostępniania tylko tego, co jest niezbędne dla użytkownika. Chroni dane przed bezpośrednim dostępem i modyfikacją z zewnątrz.

Poziomy dostępu w Pythonie:

1. Publiczne (public) - brak podkreślników

```
class Osoba:
    def __init__(self, imie):
        self.imie = imie # Publiczne - dostępne wszędzie

    def przedstaw_sie(self): # Publiczna metoda
        return f"Cześć, jestem {self.imie}"

osoba = Osoba("Anna")
print(osoba.imie) # Anna - bezpośredni dostęp OK
print(osoba.przedstaw_sie()) # Cześć, jestem Anna
```

2. Chronione (protected) - jeden podkreślnik _

```
class KontoBankowe:
    def __init__(self, wlasciciel, saldo):
        self.wlasciciel = wlasciciel
        self._saldo = saldo # Chronione - konwencja, nie dla bezpośredniego użycia
    def _waliduj_kwote(self, kwota): # Chroniona metoda
        return kwota > 0

    def wpłata(self, kwota):
        if self._waliduj_kwote(kwota):
            self._saldo += kwota
            return True
        return False

    def pobierz_saldo(self):
        return self._saldo

konto = KontoBankowe("Jan", 1000)
print(konto.pobierz_saldo()) # 1000 - OK przez metodę publiczną

# Technicznie możliwe, ale nie zalecane (konwencja):
print(konto._saldo) # 1000 - działa, ale narusza konwencję
```

3. Prywatne (private) - dwa podkreślniki __

```
class Sejf:
    def __init__(self, kod_pin):
        self.__kod_pin = kod_pin # Prywatne - name mangling
        self.__zawartosc = []

    def __sprawdz_kod(self, kod): # Prywatna metoda
        return kod == self.__kod_pin

def otworz(self, kod):
    if self.__sprawdz_kod(kod):
        return f"Sejf otwarty. Zawartość: {self.__zawartosc}"
    return "Błędny kod PIN!"
```

```
def dodaj_przedmiot(self, kod, przedmiot):
    if self.__sprawdz_kod(kod):
        self.__zawartosc.append(przedmiot)
        return "Dodano przedmiot"
    return "Błędny kod PIN!"

sejf = Sejf("1234")
print(sejf.otworz("1234")) # Działa przez publiczną metodę

# To NIE ZADZIAŁA:
# print(sejf.__kod_pin) # AttributeError
# sejf.__sprawdz_kod("1234") # AttributeError

# Name mangling - można obejść, ale NIE POWINNO SIĘ:
# print(sejf._Sejf__kod_pin) # 1234 - możliwe, ale złamanie enkapsulacji
```

4. Właściwości (Properties) - gettery i settery

W wielu językach (Java, C#) pisze się:

```
getPensja()      // getter
setPensja(5000)  // setter
```

W Pythonie robi się to zupełnie inaczej – i jest to **świadoma, filozoficzna decyzja** języka.

Główny powód: Pythonic way – „Uniform Access Principle”

Zasada jednolitego dostępu mówi:

Klient klasy **nie powinien widzieć różnicy** między:

- zwykłym atrybutem (`pracownik.pensja`)
- a obliczaną/walidowaną wartością (`pracownik.pensja`)

Dzięki `@property` możesz **zmienić implementację** w przyszłości **bez zmiany kodu**, który tej klasy używa.

Porównanie: bez property vs z property

1. Wersja „stara”

```
class Pracownik:
    def __init__(self, imie, pensja):
        self.__imie = imie
        self.__pensja = pensja

    def get_pensja(self):
        return self.__pensja

    def set_pensja(self, nowa_pensja):
        if nowa_pensja < 0:
            raise ValueError("Pensja nie może być ujemna!")
```

```
self.__pensja = nowa_pensja
```

```
p = Pracownik("Marek", 5000)
print(p.get_pensja())      # trzeba pamiętać o nawiasach i "get_"
p.set_pensja(6000)         # trzeba pamiętać o "set_"
```

2. Wersja z @property

```
class Pracownik:
    def __init__(self, imie, pensja):
        self.__imie = imie
        self.__pensja = pensja

    @property
    def pensja(self):
        return self.__pensja

    @pensja.setter
    def pensja(self, nowa_pensja):
        if nowa_pensja < 0:
            raise ValueError("Pensja nie może być ujemna!")
        if nowa_pensja < 3000:
            raise ValueError("Pensja nie może być niższa niż minimalna!")
        self.__pensja = nowa_pensja
```

```
p = Pracownik("Marek", 5000)
print(p.pensja)      # wygląda jak zwykły atrybut!
p.pensja = 6000      # też wygląda jak zwykły atrybut!
```

Kluczowa korzyść:

Jeśli w przyszłości dodasz walidację, logowanie, przeliczanie waluty itp. – **nie musisz zmieniać żadnego wiersza kodu u użytkowników Twojej klasy.**

Lekcja 17

Temat: Biblioteka standardowa: Math, random, datetime – przykłady użycia.

Biblioteka standardowa Pythona to zbiór modułów dostarczanych wraz z Pythonem, które nie wymagają instalacji. Oto przegląd najważniejszych modułów:

1. Math - operacje matematyczne

Moduł `math` dostarcza funkcje matematyczne do pracy z liczbami rzeczywistymi.

```
import math

# Stałe matematyczne
print(math.pi) # 3.141592653589793
print(math.e) # 2.718281828459045
print(math.inf) # infinity (nieskończoność)

# Funkcje podstawowe
print(math.sqrt(16)) # 4.0 - pierwiastek kwadratowy
print(math.pow(2, 3)) # 8.0 - potęgowanie
print(math.factorial(5)) # 120 - silnia

# Zaokrąglanie
print(math.ceil(4.3)) # 5 - zaokrąglenie w górę
print(math.floor(4.7)) # 4 - zaokrąglenie w dół
print(math.trunc(4.9)) # 4 - obcięcie części dziesiętnej

# Funkcje trygonometryczne
print(math.sin(math.pi/2)) # 1.0 - sinus
print(math.cos(0)) # 1.0 - cosinus
print(math.tan(math.pi/4)) # 1.0 - tangens

# Konwersje kątów
print(math.degrees(math.pi)) # 180.0 - radiany na stopnie
print(math.radians(180)) # 3.141592... - stopnie na radiany

# Logarytmy
print(math.log(10)) # 2.302585... - logarytm naturalny
print(math.log10(100)) # 2.0 - logarytm dziesiętny
print(math.log2(8)) # 3.0 - logarytm dwójkowy

# Wartość bezwzględna i znak
print(math.fabs(-5.5)) # 5.5 - wartość bezwzględna
print(math.copysign(5, -1)) # -5.0 - kopiuje znak

# GCD (największy wspólny dzielnik) i inne
print(math.gcd(48, 18)) # 6
print(math.isnan(float('nan'))) # True - sprawdza czy NaN
print(math.isinf(math.inf)) # True - sprawdza czy nieskończoność
```

2. Random - generowanie liczb losowych

```
import random
```

```

# Losowa liczba zmiennoprzecinkowa z zakresu [0.0, 1.0)
print(random.random()) # np. 0.8734523...

# Losowa liczba całkowita z zakresu
print(random.randint(1, 10)) # np. 7 (włącznie z 1 i 10)
print(random.randrange(0, 100, 5)) # np. 45 (co 5, bez 100)

# Losowa liczba zmiennoprzecinkowa z zakresu
print(random.uniform(1.5, 10.5)) # np. 7.234...

# Losowy wybór z listy
kolory = ['czerwony', 'niebieski', 'zielony', 'żółty']
print(random.choice(kolory)) # np. 'zielony'

# Losowe próbki (bez powtórzeń)
print(random.sample(kolory, 2)) # np. ['czerwony', 'żółty']

# Losowe próbki z powtórzeniami
print(random.choices(kolory, k=3)) # np. ['czerwony', 'czerwony', 'zielony']

# Mieszanie listy (in-place)
karty = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(karty)
print(karty) # np. [3, 7, 1, 9, 2, 5, 10, 4, 6, 8]

# Rozkład normalny (Gausa)
print(random.gauss(0, 1)) # średnia=0, odchylenie=1

# Ustawienie ziarna (dla powtarzalnych wyników)
random.seed(42) print(random.random()) # Zawsze ta sama wartość przy seed=42

```

3. Datetime - praca z datą i czasem

```

from datetime import datetime, date, time, timedelta

```

```

# Aktualna data i czas
teraz = datetime.now()
print(teraz) # 2026-03-30 15:30:45.123456

```

```

# Tylko data
dzisiaj = date.today()
print(dzisiaj) # 2026-03-30

```

```

# Tworzenie konkretnej daty
urodziny = date(1990, 5, 15)
print(urodziny) # 1990-05-15

```

```

# Tworzenie konkretnego czasu
godzina = time(14, 30, 0)
print(godzina) # 14:30:00

```

```

# Komponenty daty/czasu
print(f'Rok: {teraz.year}')
print(f'Miesiąc: {teraz.month}')

```

```

print(f"Dzień: {teraz.day}")
print(f"Godzina: {teraz.hour}")
print(f"Minuta: {teraz.minute}")
print(f"Sekunda: {teraz.second}")

# Dzień tygodnia
print(f"Dzień tygodnia: {dzisiaj.weekday()}") # 0=poniedziałek, 6=niedziela print(f"Nazwa dnia: {dzisiaj.strftime('%A')}")
# Monday

# Formatowanie daty
print(dzisiaj.strftime("%d/%m/%Y")) # 30/03/2026
print(dzisiaj.strftime("%d-%m-%Y")) # 30-03-2026
print(teraz.strftime("%H:%M:%S")) # 15:30:45
print(teraz.strftime("%d %B %Y, %H:%M")) # 30 March 2026, 15:30

# Parsowanie tekstu na datę
data_tekst = "2026-12-25"
data = datetime.strptime(data_tekst, "%Y-%m-%d")
print(data) # 2026-12-25 00:00:00

# Operacje na datach - timedelta
jeden_dzien = timedelta(days=1)
jutro = dzisiaj + jeden_dzien
wczoraj = dzisiaj - jeden_dzien
print(f"Jutro: {jutro}")
print(f"Wczoraj: {wczoraj}")

# Różne przedziały czasowe
tydzien = timedelta(weeks=1)
miesiac = timedelta(days=30)
godzina_delta = timedelta(hours=1)

za_tydzien = teraz + tydzien
print(f"Za tydzień: {za_tydzien}")

# Różnica między datami
roznica = date(2026, 12, 31) - dzisiaj
print(f"Do końca roku zostało {roznica.days} dni")

# Porównywanie dat
data1 = date(2026, 1, 1)
data2 = date(2026, 12, 31)
print(data1 < data2) # True

```

Lekcja 18 - Zapowiedzieć sprawdzian

Temat: List comprehensions, generatory i wyrażenia generatorowe. Match

1. List comprehensions (wyrażenia listowe)

To najwygodniejszy i najbardziej „pythoniczny” sposób tworzenia list w jednej linii.

Składnia podstawowa:

[wyrażenie **for** element **in** iterable **if** warunek]

Przykłady:

Prosty przykład

```
kwadraty = [x**2 for x in range(10)]  
print(kwadraty)           # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Z warunkiem (tylko parzyste)

```
parzyste_kwadraty = [x**2 for x in range(10) if x % 2 == 0]  
print(parzyste_kwadraty)  # [0, 4, 16, 36, 64]
```

Złożone wyrażenie

```
imiona = ["ania", "tomek", "natalia", "piotr"]  
duze_imiona = [imie.capitalize() for imie in imiona if len(imie) > 3]  
print(duze_imiona)       # ['Ania', 'Tomek', 'Natalia', 'Piotr']
```

Zagnieżdżone list comprehensions (często używane do macierzy):

Macierz 3x4 wypełniona zerami

```
macierz = [[0 for _ in range(4)] for _ in range(3)]  
print(macierz)  
# [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

Tabliczka mnożenia

```
tabliczka = [[i * j for j in range(1, 11)] for i in range(1, 11)]
```

Zalety list comprehension:

- Bardzo czytelne
- Szybsze niż pętla for + append()
- Zajmuje mniej miejsca

Wady:

- Tworzy całą listę w pamięci od razu → problem przy bardzo dużych danych.

2. Generatory (funkcje z yield)

Generator to funkcja, która zamiast *return* używa *yield*. Zwraca wartości „po kawałku” (po jednej na raz)

Różnica: **return** vs **yield**

zwykła funkcja:

```
def numbers():  
    return [1, 2, 3]
```

zwraca wszystko naraz

generator:

```
def numbers():  
    yield 1  
    yield 2  
    yield 3
```

Gdy funkcja napotka *yield*, zatrzymuje swoje wykonanie i zwraca wartość. Przy następnym wywołaniu *next()* kontynuuje dokładnie od tego miejsca

```
gen = numbers()
```

```
print(next(gen))  
print(next(gen))  
print(next(gen))
```

```
for liczba in numbers():  
    print(liczba)
```

3. Generator expressions (wyrażenia generatorowe)

To „leniwa” wersja list comprehension. Zamiast `[]` używamy `()`.

```
# List comprehension (tworzy całą listę od razu)  
kwadraty_list = [x**2 for x in range(1000000)]
```

```
# Generator expression (tworzy generator – prawie zero pamięci)  
kwadraty_gen = (x**2 for x in range(1000000))
```


List comprehension ([]) - zachłanny (eager)

☞ tworzy całą listę od razu w pamięci

```
numbers = [x * 2 for x in range(5)]  
print(numbers)
```

☞ wynik:

```
[0, 2, 4, 6, 8]
```

☞ wszystko policzone natychmiast

Generator expression (()) - leniwy (lazy)

☞ nie liczy wszystkiego od razu

☞ generuje wartości dopiero kiedy są potrzebne

```
numbers = (x * 2 for x in range(5))  
print(numbers)
```

☞ wynik:

```
<generator object ...>
```

☞ jeszcze nic nie zostało policzone!

Generator działa dopiero przy użyciu:

```
for x in numbers:  
    print(x)
```

☞ albo:

```
next(numbers)
```

Zastosowanie

- Małe dane (do kilkudziesięciu tysięcy elementów) → możesz używać **list comprehension** (wygodniej).
- Duże dane lub kiedy przetwarzasz tylko raz → zdecydowanie **generator expression**.

match to nowoczesna **instrukcja porównywania wzorców** (pattern matching), dodana w Pythonie 3.10. Często nazywana jest „switch na sterydach”, bo jest o wiele potężniejsza niż klasyczny switch z innych języków.

Składnia:

match wartosc

case wzorzec1:

 # kod gdy pasuje wzorzec1

```
case wzorzec2:
    # kod gdy pasuje wzorzec2
case _:
    # domyślny przypadek (jak else)
```

Przykład - ocena szkolna

```
ocena = "B"
```

```
match ocena:
    case "A":
        print("Celujący!")
    case "B":
        print("Bardzo dobry")
    case "C":
        print("Dobry")
    case "D":
        print("Dostateczny")
    case "E" | "F":          # | - or bitowy
        print("Niedostateczny")
    case _:
        print("Nieprawidłowa ocena")
```

Lekcja 19

Temat: Powtórzenie z list, krotek, słowników, zbiorów oraz ich użycia wewnątrz klas

1. Mutowalność (mutable) vs niemutowalność (immutable)

Typ	Mutowalność	Czy można zmienić po utworzeniu?
list	mutowalna	✅ można dodawać, usuwać, zmieniać elementy
dict	mutowalny	✅ można dodawać, usuwać, zmieniać pary klucz-wartość
set	mutowalny	✅ można dodawać, usuwać elementy
tuple	niemutowalna	❌ NIE można zmienić! Jest jak "zamrożona lista"

1. List (lista) – MUTOWALNA

```
my_list = [1, 2, 3]
```

```
my_list[0] = 100      # zmiana elementu  
my_list.append(4)     # dodanie elementu
```

```
print(my_list)        # [100, 2, 3, 4]
```

2. Dict (słownik) – MUTOWALNY

```
my_dict = {"a": 1, "b": 2}
```

```
my_dict["a"] = 100    # zmiana wartości  
my_dict["c"] = 3      # dodanie nowego klucza
```

```
print(my_dict)        # {'a': 100, 'b': 2, 'c': 3}
```

3. Set (zbiór) – MUTOWALNY

```
my_set = {1, 2, 3}
```

```
my_set.add(4)  
my_set.remove(2)
```

```
print(my_set)         # {1, 3, 4}
```

4. Tuple (krotka) – NIEMUTOWALNA

```
my_tuple = (1, 2, 3)
```

```
# my_tuple[0] = 100    # ✗ BŁĄD
```

Ważny przypadek (tuple z listą w środku)

```
my_tuple = (1, [2, 3])
```

```
my_tuple[1].append(4)
```

```
print(my_tuple)        # (1, [2, 3, 4])
```

Krotka jest niemutowalna, ale **obiekt w środku (lista)** już nie.

2. Podstawowe metody dla każdego typu

LISTA (mutowalna)

```
lista = [1, 2, 3]

# Dodawanie
lista.append(4)      # [1, 2, 3, 4] - dodaje na koniec
lista.insert(1, 99)  # [1, 99, 2, 3, 4] - wstawia na pozycję
lista.extend([5, 6]) # [1, 99, 2, 3, 4, 5, 6] - łączy listy

# Usuwanie
lista.remove(99)     # [1, 2, 3, 4, 5, 6] - usuwa pierwsze wystąpienie
element = lista.pop() # usuwa i zwraca ostatni (6)
lista.pop(0)         # usuwa i zwraca element z indeksu 0 (1)
lista.clear()        # [] - usuwa wszystko

# Inne przydatne
lista = [3, 1, 4, 1, 5]
lista.sort()         # [1, 1, 3, 4, 5] - sortuje
lista.reverse()      # [5, 4, 3, 1, 1] - odwraca
len(lista)           # 5 - długość
3 in lista           # True - sprawdza czy istnieje
```

KROTKA (niemutowalna) - mało metod

```
krotka = (1, 2, 3, 2, 4)

# Nie można dodawać ani usuwać!
# Ale można odczytywać:
print(krotka[0])     # 1
print(krotka.count(2)) # 2 - ile razy występuje
print(krotka.index(3)) # 2 - na którym indeksie jest 3
print(len(krotka))   # 5

# Jedyne sposoby "dodania" - stworzenie nowej
nowa = krotka + (5, 6) # (1, 2, 3, 2, 4, 5, 6)
```

SŁOWNIK (mutowalny)

```
słownik = {"a": 1, "b": 2}

# Dodawanie / zmiana
```

```

sownik["c"] = 3          # {"a":1, "b":2, "c":3}
sownik.update({"d": 4}) # dodaje wiele

# Usuwanie
del sownik["b"]          # {"a":1, "c":3, "d":4}
sownik.pop("c")          # usuwa i zwraca wartosc 3 dla klucza c
sownik.popitem()         # usuwa i zwraca ostatnią parę ('d', 4)
sownik.clear()           # {}

# Dostęp
print(sownik.get("a", 0)) # 1 (bez błędu jeśli nie ma)
print(sownik.keys())      # widok kluczy
print(sownik.values())    # widok wartości
print(sownik.items())     # widok par (klucz, wartość)

```

ZBIÓR (mutowalny) - elementy unikalne, nieuporządkowane

```

zbior = {1, 2, 3}

# Dodawanie
zbior.add(4)          # {1, 2, 3, 4}
zbior.add(2)          # {1, 2, 3, 4} - nic się nie dzieje (już jest)

# Usuwanie
zbior.remove(3)       # {1, 2, 4} - błąd jeśli nie ma
zbior.discard(10)     # {1, 2, 4} - brak błędu jeśli nie ma
zbior.pop()           # usuwa i zwraca jakiś element (nie wiadomo który)
zbior.clear()         # {}

# Operacje na zbiorach
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B)          # {1,2,3,4,5} - suma
print(A & B)          # {3} - przecięcie
print(A - B)          # {1,2} - różnica

```

3. Proste klasy z zastosowaniem każdego typu

Klasa 1: ListaZakupow - używa LISTY

```

class ListaZakupow:
    def __init__(self):
        self.produkty = [] # lista przechowuje produkty

```

```

def dodaj(self, produkt):
    self.produkty.append(produkt)
    print(f"Dodano: {produkt}")

def usun(self, produkt):
    if produkt in self.produkty:
        self.produkty.remove(produkt)
        print(f"Usunięto: {produkt}")
    else:
        print(f"Brak {produkt} na liście")

def wyswietl(self):
    if not self.produkty:
        print("Lista zakupów jest pusta")
    else:
        print("LISTA ZAKUPÓW:")
        for i, produkt in enumerate(self.produkty, 1):
            print(f" {i}. {produkt}")

def ile_produktow(self):
    return len(self.produkty)

# Użycie:
lista = ListaZakupow()
lista.dodaj("chleb")
lista.dodaj("mleko")
lista.dodaj("jajka")
lista.wyswietl()
lista.usun("mleko")
lista.wyswietl()
print(f"Liczba produktów: {lista.ile_produktow()}")

```

Klasa 2: Punkt2D - używa KROTKI (niemutowalna)

```

class Punkt2D:
    def __init__(self, x, y):
        # krotka przechowuje współrzędne (nie mogą się zmienić)
        self.wspolrzedne = (x, y)

    def pobierz_x(self):
        return self.wspolrzedne[0]

    def pobierz_y(self):

```

```

        return self.wspolrzedne[1]

def odleglosc_od_poczatku(self):
    x, y = self.wspolrzedne
    return (x**2 + y**2) ** 0.5

def __str__(self):
    return f"Punkt{self.wspolrzedne}"

# Nie ma metody do zmiany współrzędnych! (bo krotka jest niemutowalna)
# Aby zmienić, trzeba stworzyć nowy obiekt

# Użycie:
p1 = Punkt2D(3, 4)
print(p1) # Punkt(3, 4)
print(f"x = {p1.pobierz_x()}") # x = 3
print(f"Odległość: {p1.odleglosc_od_poczatku():.2f}") # 5.00

# Nie można zrobić: p1.wspolrzedne[0] = 5 (błąd!)
# Aby zmienić, tworzymy nowy punkt:
p2 = Punkt2D(5, 6)

```

Klasa 3: Telefon - używa SŁOWNIKA

```

class Telefon:
    def __init__(self):
        # słownik: nazwa kontaktu -> numer telefonu
        self.kontakty = {}

    def dodaj_kontakt(self, nazwa, numer):
        self.kontakty[nazwa] = numer
        print(f"Dodano: {nazwa} -> {numer}")

    def usun_kontakt(self, nazwa):
        if nazwa in self.kontakty:
            usuniety = self.kontakty.pop(nazwa)
            print(f"Usunięto: {nazwa} ({usuniety})")
        else:
            print(f"Brak kontaktu: {nazwa}")

    def zadzwon(self, nazwa):
        if nazwa in self.kontakty:
            print(f"Dzwonię do {nazwa} pod numer {self.kontakty[nazwa]}")
        else:
            print(f"Nie mam numeru do {nazwa}")

    def wyswietl_kontakty(self):

```

```

    if not self.kontakty:
        print("Brak kontaktów")
    else:
        print("KONTAKTY:")
        for nazwa, numer in self.kontakty.items():
            print(f" {nazwa}: {numer}")

def znajdz_po_numerze(self, numer):
    # szukamy kto ma dany numer
    for nazwa, nr in self.kontakty.items():
        if nr == numer:
            return nazwa
    return None

# Użycie:
telefon = Telefon()
telefon.dodaj_kontakt("Ania", "123-456-789")
telefon.dodaj_kontakt("Kamil", "987-654-321")
telefon.wyswietl_kontakty()
telefon.zadzwon("Ania")
kto = telefon.znajdz_po_numerze("987-654-321")
print(f"Numer 987-654-321 należy do: {kto}")

```

Klasa 4: LiczbyUnikalne - używa ZBIORU

```

class LiczbyUnikalne:
    def __init__(self):
        self.liczby = set() # zbiór przechowuje unikalne liczby

    def dodaj(self, liczba):
        if liczba in self.liczby:
            print(f"{liczba} już jest w zbiorze!")
        else:
            self.liczby.add(liczba)
            print(f"Dodano: {liczba}")

    def usun(self, liczba):
        if liczba in self.liczby:
            self.liczby.remove(liczba)
            print(f"Usunięto: {liczba}")
        else:
            print(f"{liczba} nie ma w zbiorze")

    def wyswietl(self):
        if not self.liczby:
            print("Zbiór jest pusty")
        else:
            print(f"Zbiór liczb: {sorted(self.liczby)}") # sortujemy dla czytelności

```



```

def ile_liczb(self):
    return len(self.liczby)

def czy_zawiera(self, liczba):
    return liczba in self.liczby

def polacz(self, inny_zbior):
    """łączy z innym zbiorem (suma)"""
    self.liczby.update(inny_zbior)
    print("Połączono zbiory")

# Użycie:
zb = LiczbyUnikalne()
zb.dodaj(5)
zb.dodaj(10)
zb.dodaj(5)      # komunikat: już jest!
zb.dodaj(7)
zb.wyswietl()    # {5, 7, 10}
zb.usun(10)
zb.wyswietl()    # {5, 7}
print(f"Czy jest 5? {zb.czy_zawiera(5)}")

```

4. Ćwiczenia dla uczniów

Ćwiczenie 1 (Lista)

Dodaj do ListaZakupow metodę usun_ostatni() która usuwa ostatni produkt z listy.

Ćwiczenie 2 (Krotka)

Stwórz klasę KartaPlatnicza która przechowuje w krotce: numer karty, datę ważności, kod CVV. Nie dodawaj metody do zmiany tych danych.

Ćwiczenie 3 (Słownik)

Dodaj do Telefon metodę edytuj_numer(nazwa, nowy_numer) która zmienia numer istniejącego kontaktu.

Ćwiczenie 4 (Zbiór)

Dodaj do `LiczyUnikalne` metodę `czesc_wspolna(inny_zbior)` która zwraca nowy obiekt `LiczyUnikalne` zawierający tylko liczby występujące w obu zbiorach.

Lekcja 20

Temat: Rozwiązania interfejsów w Python za pomocą: Protokoły (duck typing), modułu `abc` (Abstract Base Classes), modułu `typing.Protocol`.

W Pythonie nie ma „prawdziwych” interfejsów jak np. w Java czy TypeScript, tylko bardziej luźne podejście (duck typing, klasy abstrakcyjne itd.).

Przykład interface w TypeScript

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
const user: User = {  
  id: 1,  
  name: "Jan",  
  email: "jan@example.com"  
};
```

Interface z metodą

```
interface Animal {  
  makeSound(): void;  
}  
  
class Dog implements Animal {  
  makeSound(): void {  
    console.log("Hau hau");  
  }  
}
```

Python jest językiem dynamicznym i używa mechanizmu zwanego **protokołami (duck typing)** oraz od wersji 3.3+ — **modułu abc (Abstract Base Classes)**, a od Pythona 3.8+ — **modułu typing.Protocol**.

Duck Typing (dosłownie: „kacze typowanie”) to podstawowa filozofia Pythona, jak sprawdza, czy obiekt nadaje się do użycia.

Pochodzi od powiedzenia:

„Jeśli coś chodzi jak kaczka, pływa jak kaczka i kwacze jak kaczka — to jest kaczka.”

W programowaniu oznacza to:

Nie interesuje nas, z jakiej klasy jest obiekt i po czym dziedziczy. Liczy się tylko to, **czy ma odpowiednie metody i atrybuty**, których potrzebujemy.

Jeśli ma — Python traktuje go jako „właściwy typ”.

1. Najważniejsze sposoby definiowania „interfejsu” w Pythonie

A) Prosty przykład Duck Typing (bez żadnego Protocol ani ABC)

```
class Kaczka:
    def kwacz(self):
        print("Kwak kwak!")

class Samolot:
    def kwacz(self):
        print("Bzzzzz... (udaję kaczkę)")

def wydaj_dzwiek(obiekt):
    obiekt.kwacz()    # Python nie sprawdza typu – tylko czy ma metodę kwacz()

wydaj_dzwiek(Kaczka())
wydaj_dzwiek(Samolot())
```

To jest czysty duck typing — działa, bo obie klasy mają metodę kwacz()

B) Duck Typing + Protocol (statyczne duck typing)

Od Pythona 3.8 mamy Protocol, który pozwala **opisać** taki „zestaw zachowań” i sprawdzać go narzędziami do typów (mypy, Pyright, VS Code itp.).

```
from typing import Protocol
```

```
class Kwaczaca(Protocol):      # To jest definicja „interfejsu” / protokołu
    def kwacz(self) -> None: ...
```

```
def wydaj_dzwiek(obiekt: Kwaczaca) -> None:    # oczekujemy obiektu, który „kwacze”
    obiekt.kwacz()
```

```
wydaj_dzwiek(Kaczka())
wydaj_dzwiek(Samolot())
```

Kluczowa różnica:

- **Czysty duck typing** → sprawdza tylko w trakcie działania programu (runtime). Jeśli metody nie ma — błąd w trakcie uruchomienia.
- **Protocol** → dodaje **statyczne sprawdzanie typów** (przed uruchomieniem), ale **nadal nie wymaga dziedziczenia**.

Przykład 2

```
from typing import Protocol
```

```
class Drawable(Protocol):
    """Protokół definiujący obiekty, które można narysować."""
```

```
    def draw(self) -> None: ...
    def get_area(self) -> float: ...
```

```
class Circle:
    """Klasa reprezentująca koło - implementuje protokół Drawable."""
```

```
    def __init__(self, radius: float):
        self.radius = radius

    def draw(self) -> None:
        print(f"Rysuję koło o promieniu {self.radius}")

    def get_area(self) -> float:
        return 3.14 * self.radius ** 2
```

```
class Rectangle:
    """Klasa reprezentująca prostokąt - implementuje protokół Drawable."""
```

```
    def __init__(self, width: float, height: float):
        self.width = width
```

```

        self.height = height

def draw(self) -> None:
    print(f"Rysuję prostokąt {self.width}x{self.height}")

def get_area(self) -> float:
    return self.width * self.height

def render(shape: Drawable) -> None:
    shape.draw()
    print(f"Pole: {shape.get_area():.2f}\n")

if __name__ == "__main__":
    render(Circle(5))
    render(Rectangle(4, 6))

```

C) Abstract Base Class ABC – starszy sposób

Przykład: Abstract Base Class (ABC)

```

from abc import ABC, abstractmethod

class Drawable(ABC):
    """Abstrakcyjna klasa bazowa (interfejs) definiująca obiekty, które można
    narysować."""

    @abstractmethod
    def draw(self) -> None:
        """Metoda odpowiedzialna za rysowanie figury."""
        pass

    @abstractmethod
    def get_area(self) -> float:
        """Metoda zwracająca pole figury."""
        pass

class Circle(Drawable):
    # ← musimy dziedziczyć po Drawable
    """Klasa reprezentująca koło - implementuje interfejs Drawable."""

    def __init__(self, radius: float):
        self.radius = radius

    def draw(self) -> None:

```

```

        print(f"Rysuję koło o promieniu {self.radius}")

def get_area(self) -> float:
    return 3.14 * self.radius ** 2

class Rectangle(Drawable):    # ← również musimy dziedziczyć po Drawable
    """Klasa reprezentująca prostokąt - implementuje interfejs Drawable."""

    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def draw(self) -> None:
        print(f"Rysuję prostokąt {self.width}x{self.height}")

    def get_area(self) -> float:
        return self.width * self.height

def render(shape: Drawable) -> None:
    """Funkcja oczekuje obiektu, który dziedziczy po Drawable (ABC)."""
    shape.draw()
    print(f"Pole: {shape.get_area():.2f}\n")

if __name__ == "__main__":
    render(Circle(5))
    render(Rectangle(4, 6))

```

Lekcja 21

Temat: Ćwiczenia praktyczne z klas i dziedziczenia

Zadanie 1

```

'''
Zadanie 1 – Klasy (podstawy)
Treść

```

Stwórz klasę Produkt, która:

posiada pola:

```
nazwa
cena
_iloc (protected)
```

posiada metody:

```
dodaj_sztuki(ile)
sprzedaj(ile) – nie pozwala sprzedać więcej niż jest
wartosc_magazynu() → cena * ilość
```

używa @property do odczytu ilości

```
'''
'''
```

Podpowiedź

```
class Produkt:
```

```
    def __init__(self, nazwa, cena, ilosc):
```

```
        @property
```

```
        def ilosc(self):
            return self._iloc
```

```
    def dodaj_sztuki(self, ile):
```

```
    def sprzedaj(self, ile):
        if ile > self._iloc:
            raise ValueError("Brak wystarczającej ilości")
        self._iloc -= ile
```

```
    def wartosc_magazynu(self):
```

```
'''
```

Zadanie 2

```
'''
```

Zadanie 2 – Dziedziczenie + ABC + typ jako parametr

Treść

Stwórz:

Abstrakcyjną klasę Figura (ABC) z metodą:

```
    pole()
```

Klasy:

```
    Prostokat
```

Kolo

Klasę KalkulatorFigur, która ma metodę:

```
def oblicz_pole(figura: Figura)
```

→ przyjmuje obiekt klasy Figura

```
'''  
'''
```

Podpowiedź :

```
from abc import ABC, abstractmethod  
import math
```

```
class Figura(ABC):  
    @abstractmethod  
    def pole(self):  
        pass
```

```
class Prostokat(Figura):  
    def __init__(self, a, b):  
        self.a = a  
        self.b = b
```

```
class Kolo(Figura):
```

```
class KalkulatorFigur:  
    '''
```

Zadanie 3

```
'''  
Zadanie 1 – Klasy (podstawy)  
Treść
```

Stwórz klasę Produkt, która:

posiada pola:

```
    nazwa  
    cena  
    _ilosc (protected)
```

posiada metody:

```
    dodaj_sztuki(ile)  
    sprzedaj(ile) – nie pozwala sprzedać więcej niż jest
```



```
wartosc_magazynu() → cena * ilość
```

używa @property do odczytu ilości

```
'''  
'''
```

Podpowiedź

```
class Produkt:
```

```
    def __init__(self, nazwa, cena, ilosc):
```

```
        @property
```

```
        def ilosc(self):
```

```
            return self._ilosc
```

```
    def dodaj_sztuki(self, ile):
```

```
    def sprzedaj(self, ile):
```

```
        if ile > self._ilosc:
```

```
            raise ValueError("Brak wystarczającej ilości")
```

```
        self._ilosc -= ile
```

```
    def wartosc_magazynu(self):
```

```
'''
```